Artificial Intelligence Laboratory

Lab Experiment 4 — A* (A-Star) Algorithm

6th Semester · B.Tech Computer Science & Engineering · NIT Srinagar

Student Name		**Enrollment No.**	
Batch		**Date**	

Instructions: Answer all 4 scenario-based problems. For each problem: (a) build the graph/state representation, (b) fill the A* open-list trace table, (c) write Python code using `heapq`, and (d) answer the analysis questions. A* uses $f(n) = g(n) + h(n)$. Always pop the node with **lowest** $f(n)$.

Quick Reference — A* Algorithm

Property	Value	Reason
Evaluation function	$f(n) = g(n) + h(n)$	Actual cost + heuristic estimate
$g(n)$	Actual cost: start $\rightarrow n$	Tracked as we expand nodes
$h(n)$	Heuristic: $n \rightarrow$ goal	Must be admissible for optimality
Data structure	Min-Heap on $f(n)$	`heapq` in Python
Admissible heuristic	$h(n) \leq h^*(n)$	Never overestimates true cost
Complete?	Yes	Finds solution if one exists
Optimal?	Yes (admissible h)	Guaranteed shortest/cheapest path
Time complexity	$O(b^d)$	b = branching factor, d = depth
Space complexity	$O(b^d)$	Stores all generated nodes
vs Best-First	BFS* uses only $h(n)$	A* adds $g(n)$, so it is optimal
Python: push	`heapq.heappush(open_list, (f, g, node, path))`	
Python: pop	`heapq.heappop(open_list)` gives lowest f	

1

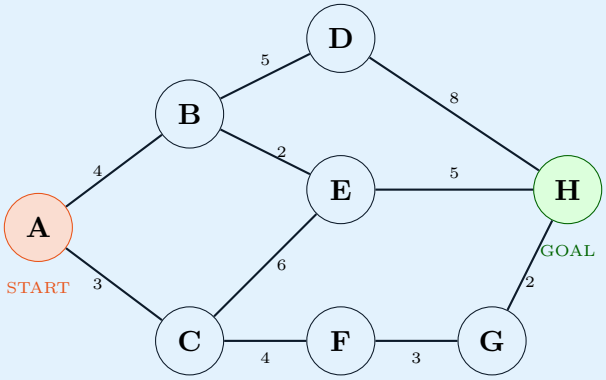
Scenario: Optimal Route Between Cities

A smart navigation app has a road network of **8 cities** (A–H). A driver must travel from city **A** to city **H**. Unlike Best First Search, the app uses **A*** to guarantee the **cheapest** total route, not just the one that looks closest.

Road network (city – city : road distance km):
A–B(4), A–C(3), B–D(5), B–E(2), C–E(6), C–F(4), D–H(8), E–H(5), F–G(3), G–H(2)

Heuristic $h(n)$ = straight-line distance to H:

City	A	B	C	D	E	F	G	H
$h(n)$ (km)	10	8	7	5	4	6	2	0



Tricky part: Best First Search (Lab 3) found path A→C→E→H = 14 km. A* should find the true cheapest path by tracking $g(n)$ as well. Which path does A* find? Is it different from Best First Search?

Tasks:

1. For each node below, write $g(n)$, $h(n)$, and $f(n)$ when A* first reaches it from A. **[4 Marks]**
2. Fill the A* open-list trace table (expand until H is popped). **[6 Marks]**
3. Write Python A* code using `heapq`. Push `(f, g, node, path)`. **[8 Marks]**
4. **Compare:** What path did Best First Search find (Lab 3)? What does A*

find? Why are they different? **[2 Marks]**

(a) g, h, f values when first reached:

Node	$g(n)$ (actual cost from A)	$h(n)$ (given)	$f(n) = g + h$	Reached via
B		8		
C		7		
D		5		
E		4		
F		6		
G		2		
H		0		

(b) A* Open-List Trace (pop lowest f each step):

Step	Pop node	g	$f = g + h$	Open list (node:f) after pop
0	—	—	—	A:10
1				
2				
3				
4				
5				
6				

Optimal path: _____ **Total cost:** _____ km

(c) Python Code:

(d) Comparison with Best First Search:

1

Scenario: Warehouse Robot Shortest Route

A warehouse robot must navigate a **5×5 grid** from the top-left corner **(0,0)** to the bottom-right corner **(4,4)**. Cells marked `1` are blocked shelving units. The robot moves Up, Down, Left, Right. Each move costs **1 unit**.

Grid (0 = passable, 1 = blocked):

0	0	1	0	0
0	1	0	0	0
0	0	0	1	0
0	1	0	0	0
0	0	0	1	0

Heuristic: Manhattan distance to $(4, 4)$: $h(r, c) = |4 - r| + |4 - c|$

Tricky part: Two blocked clusters create detours. A* tracks $g(n)$ = actual steps taken so far + $h(n)$ = Manhattan distance remaining. It will always find the minimum-step path, unlike Best First Search.

Key difference from BFS* (Lab 3): In Lab 3, the grid robot used Best First Search and was *not* guaranteed to find minimum steps. A* adds $g(n)$ = steps taken so far. $f(n) = g(n) + h(n)$ ensures the minimum-cost (fewest steps) path is found.

Tasks:

1. Calculate $h(r, c)$ for: $(0, 0)$, $(2, 2)$, $(4, 3)$. **[3 Marks]**
2. Trace A* for the first **5 expansion steps** from $(0, 0)$. Show g, h, f for each cell when pushed to open list. **[6 Marks]**
3. Write complete Python A* code for the grid. Requirements: **[9 Marks]**
   - State = tuple `(row, col)`
   - Open list entry = `(f, g, (row,col), path)`
   - Skip blocked cells & out-of-bound positions
   - Print the path and total steps
4. **Analysis:** If all cells were unblocked, what would the minimum steps be from

$(0,0)$ to $(4,4)$? Does A* find it? Explain. **[2 Marks]**

(a) Heuristic values:

$h(0,0) =$ _____ $h(2,2) =$ _____ $h(4,3) =$ _____

(b) First 5 A* expansion steps:

Step	Pop cell	g	h	$f = g + h$	Neighbours pushed
0	(0,0)	0	8	8	(0,1) and (1,0)
1					
2					
3					
4					

(c) Python Code:

(d) Minimum steps analysis (unblocked grid):

Minimum steps (unblocked) = _____ Does A* find it? _____ Why? _____

1

Scenario: Sliding Tile Puzzle

The **8-puzzle** is a 3×3 grid with tiles numbered 1–8 and one blank space (0). Tiles slide into the blank. Find the **minimum number of moves** to reach the goal configuration from the given start.

Start State

1	2	5
3	4	0
6	7	8

Goal State

1	2	3
4	5	6
7	8	0

Heuristic 1 (Misplaced Tiles): $h_1(s)$ = number of tiles not in their goal position (excluding blank).
Heuristic 2 (Manhattan Distance): $h_2(s) = \sum_{\text{tile } t} |r_t - r_t^*| + |c_t - c_t^*|$ where (r_t, c_t) is current position and (r_t^*, c_t^*) is goal position.

Tricky part: Both heuristics are admissible, but $h_2 \geq h_1$ always (Manhattan is more informed). A* with h_2 expands *fewer* nodes than A* with h_1. You must implement **both** and compare.

State representation: flat tuple `(1,2,5,3,4,0,6,7,8)`, blank = index of 0.

Tasks:

1. Calculate h_1 and h_2 for the given start state. Show tile-by-tile working for h_2. **[5 Marks]**
2. Write the A* algorithm pseudocode for the 8-puzzle (state-space search). **[4 Marks]**
3. Write Python A* code that: **[9 Marks]**
   - Represents state as a tuple of 9 integers
   - Generates all valid moves (slide blank Up/Down/Left/Right)
   - Uses **Manhattan Distance** as heuristic
   - Prints move sequence and total moves

4. **Tricky:** Why is Manhattan distance **admissible**? Can $h(n) > h^*(n)$? What happens to A* if the heuristic is not admissible? **[2 Marks]**

(a) Heuristic calculations for start state `(1,2,5,3,4,0,6,7,8)`:

h_1 (misplaced tiles): Tiles out of place = _____ h_1 = _____

h_2 (Manhattan distance) tile-by-tile:

Tile	Current pos	Goal pos	$\lvert\Delta r\rvert$	$\lvert\Delta c\rvert$	Distance
1	(0,0)	(0,0)	0	0	0
2	(0,1)	(0,1)	0	0	0
5		(1,1)			
3		(0,2)			
4		(1,0)			
6		(1,2)			
7		(2,0)			
8		(2,1)			
				Total h_2	

(b) Pseudocode:

(c) Python Code:

(d) Admissibility analysis:

1

Scenario: Internet Packet Routing

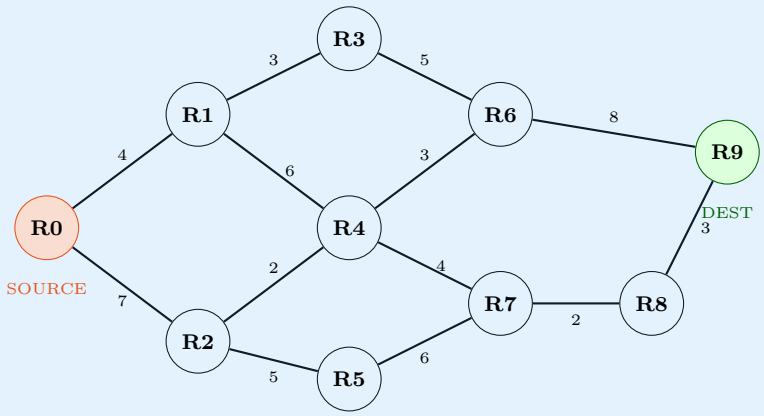
A network has **10 routers** (R0–R9). A data packet must travel from **router R0** to **router R9** with the **minimum total latency** (ms). Each link has a latency cost. The network engineer uses A* with $h(n)$ = estimated remaining latency to R9 (from network topology knowledge).

Network links (router – router : latency ms):
R0–R1(4), R0–R2(7), R1–R3(3), R1–R4(6), R2–R4(2), R2–R5(5),
R3–R6(5), R4–R6(3), R4–R7(4), R5–R7(6), R6–R9(8),
R7–R8(2), R8–R9(3)

Heuristic $h(n)$ = estimated latency to R9:

Router	R0	R1	R2	R3	R4	R5	R6	R7	R8	R9
$h(n)$ (ms)	18	14	12	10	8	10	7	5	3	0

Tricky part: There are multiple plausible paths (via R6 or via R8), and the heuristic for R2 is 12 but R2 leads quickly to R4 which reaches R8 — a short, cheap final hop. A* must correctly balance $g(n)$ and $h(n)$ to find the global optimum, not just follow low-h nodes greedily.



Tasks:

1. **Identify two possible paths** from R0 to R9 (without running the algorithm) and compute their actual costs manually. **[4 Marks]**
2. Fill the A* open-list trace. Pop until R9 is expanded. **[7 Marks]**

3. Write Python A* code. Output must include: **[9 Marks]**
   - Each node expanded with its g, h, f values
   - The optimal path
   - Total latency in ms
4. **Tricky:** Is the given $h(n)$ admissible? Verify for R2: actual shortest cost R2→R9 vs $h(\text{R2}) = 12$. **[Marks]**

(a) Two candidate paths and their costs:

Path 1: _____ Cost = _____ ms

Path 2: _____ Cost = _____ ms

(b) A* Open-List Trace:

Step	Pop	g	h	f	Open list (router:f) after
0	—	—	—	—	R0:18
1					
2					
3					
4					
5					
6					
7					

Optimal path: _____ **Total latency:** _____ ms

(c) Python Code:

(d) Admissibility check for R2:

Actual shortest cost R2→R9 = _____ ms

$h(\text{R2}) = 12$ ms

Is $h(\text{R2}) \leq$ actual cost? _____ Is heuristic admissible? _____